Kotlin Cheat Sheet

One page of terms for everything taught in the Kotlin modules, ordered from the basics to the advanced topics. Examples use the ShopKart model (Product, Money, Order, Customer) and the Learnly catalogue.

Contents

1. Basics
2. Classes and objects
3. Null safety
4. Data classes and state modelling
5. Extensions, functions and lambdas
6. Collections
7. Interop with Java and Spring
8. Quick reference

1. Basics

Term	What it means	Example
`val`	Read-only variable, assigned once (like `final`)	`val name = "Laptop"`
`var`	Mutable variable, can be reassigned	`var stock = 3`
Type inference	Compiler works out the type from the value; the type is still fixed	`val price = 65000` is `Int`
Basic types	`Int`, `Long`, `Double`, `Boolean`, `Char`, `String`; all are objects in source, primitives in bytecode	`val big = 5_000_000_000L`
No implicit widening	An `Int` is not silently turned into a `Long`; convert explicitly	`val l: Long = i.toLong()`
Function	Declared with `fun`; return type after the parameter list	`fun total(a: Int, b: Int): Int { return a + b }`
Default argument	A parameter with a fallback value	`fun badge(t: String, status: String = "OPEN")`
Named argument	Pass by name, in any order	`badge(status = "DONE", t = "Guide")`
Single-expression function	Body is one expression after `=`	`fun double(x: Int) = x * 2`
`Unit`	"Returns nothing" (the Kotlin `void`)	`fun log(s: String): Unit`
String template	Insert values with `$` or `${}`	`"Total: $total, tax: ${total / 10}"`
Multiline string	Triple quotes, keeps line breaks; `trimIndent()` removes margin	`"""Line 1\n Line 2""".trimIndent()`
`if` as expression	`if` returns a value, so no ternary operator	`val label = if (qty > 0) "In stock" else "Sold out"`
`when`	Powerful switch; works as a statement or an expression	`when (qty) { 0 -> "none"; in 1..5 -> "low"; else -> "ok" }`
`try` as expression	`try`/`catch` returns a value	`val n = try { s.toInt() } catch (e: NumberFormatException) { 0 }`
Range	`..` inclusive, `until` excludes the end, `downTo`, `step`	`1..5`, `0 until 5`, `10 downTo 0 step 2`
`in` / `!in`	Membership or range test	`if (x in 1..10)`
`for` loop	Iterates over anything iterable	`for (p in products) println(p.name)`
`while` / `do-while`	Same as Java	`while (n > 0) n--`
`break` / `continue`	Loop control, work with labels	`outer@ for (...) { ... break@outer }`

Term	What it means	Example
IDE converter	IntelliJ "Convert Java File to Kotlin" gives a first draft; always clean it up	Replace getters, loops and `StringBuilder` by idioms

2. Classes and objects

Term	What it means	Example
Primary constructor	Parameters declared in the class header; `val`/`var` make them properties	`class Product(val id: Int, var name: String)`
Property	Field plus getter/setter in one declaration; Java getters/setters appear as properties	`p.name` instead of `p.getName()`
Custom getter / setter	Validation or a computed value on a property	`val isCheap get() = price < 100`
`field`	The backing field inside a custom accessor	`set(v) { field = v.trim() }`
`init` block	Runs when the object is created, after the primary constructor	`init { require(qty >= 0) }`
`require` / `check`	Throw `IllegalArgumentException` / `IllegalStateException` when a condition fails	`require(price > 0) { "price must be positive" }`
Secondary constructor	Extra constructor that calls `this(...)`	`constructor(name: String) : this(0, name)`
Visibility	`public` (default), `internal` (module), `protected`, `private`	`private val secret = "x"`
`private set`	Readable from outside, writable only inside	`var stock = 0; private set`
Final by default	Classes and members cannot be extended or overridden unless marked `open`	`class A` cannot be a parent
`open` / `override`	Allow and perform inheritance	`open class Payment { open fun fee() = 0 }`
`interface`	Contract; may hold default method bodies and abstract properties	`interface Shippable { fun ship() = println("shipping") }`
`abstract class`	Cannot be instantiated; may mix abstract and concrete members	`abstract class Payment { abstract fun pay() }`
`object`	Singleton declared in one keyword	`object Config { val region = "IN" }`
`companion object`	Holds what Java would make `static`: factories and constants	`companion object { fun of(id: Int) = Product(id, "?") }`
`const val`	Compile-time constant, only for primitives and `String`	`const val PREFIX = "KT-"`
Top-level function / constant	Declared in a file, outside any class	`fun formatInr(n: Int) = "INR $n"`
Data class (preview)	A class whose job is to carry data, replaces a Java POJO (see section 4)	`data class Customer(val id: Long, val name: String)`

3. Null safety

Term	What it means	Example
Non-null type `T`	Cannot hold `null`; checked at compile time	`var s: String = "a"` then `s = null` does not compile
Nullable type `T?`	May hold `null`; must be handled before use	`var s: String? = null`
`T` is a subtype of `T?`	A non-null value fits a nullable slot, never the reverse	`val a: String? = "x"` works, `val b: String = a` does not
Safe call `?.`	Calls only if not null, otherwise the result is `null`	`customer?.address?.city`
Chained safe calls	One line replaces nested `if` checks	`order?.customer?.name`

Term	What it means	Example
Elvis `?:`	Supplies a default when the left side is `null`	`val city = address?.city ?: "Unknown"`
Early return with Elvis	Leave the function when a value is missing	`val c = find(id) ?: return`
`throw` on the right of Elvis	Fail with a clear error	`val c = find(id) ?: throw IllegalStateException("no customer")`
`requireNotNull`	Turns `T?` into `T` or throws `IllegalArgumentException`	`val c = requireNotNull(find(id)) { "id $id missing" }`
Not-null assertion `!!`	"Trust me": throws `NullPointerException` if `null`; avoid it	`name!!.length`
Smart cast	After a null or type check the compiler treats the value as the narrower type	`if (s != null) s.length`
`is` / `!is`	Type check that also smart-casts	`if (pay is Card) pay.number`
Smart cast on `var` property	Impossible, because it could change; copy into a local `val` first	`val local = order.coupon; if (local != null) ...`
Safe cast `as?`	Cast that returns `null` instead of throwing	`val c = obj as? Customer`
`?.let { }`	Run a block only for a non-null value	`coupon?.let { apply(it) }`
Platform type `T!`	A Java type with unknown nullability; you decide, and a wrong guess crashes at runtime	`val s: String? = javaService.find()`
JetBrains annotations	`@Nullable` / `@NotNull` on Java code make Kotlin enforce nullability	`@Nullable String find()` becomes `String?`

Scope functions

Function	Object is	Returns	Use it to
`let`	`it`	lambda result	Run code only for a non-null value, or transform it
`run`	`this`	lambda result	Compute a result using the object's members
`with(obj)`	`this`	lambda result	Work with a non-null object without repeating its name
`apply`	`this`	the object	Configure an object, builder style
`also`	`it`	the object	Side effect such as logging, then keep the chain going

4. Data classes and state modelling

Term	What it means	Example
`data class`	Compiler generates `equals`, `hashCode`, `toString`, `copy`, `componentN` from the primary constructor	`data class Money(val amount: Int, val currency: String)`
`==` vs `===`	`==` calls `equals` (structural), `===` compares identity	`Money(1,"INR") == Money(1,"INR")` is `true`, `===` is `false`
Primary constructor only	Properties in the class body are ignored by the generated members	Body properties are not in `toString` or `equals`
`copy()`	New object with some fields changed, the original is untouched	`order.copy(status = "PAID")`
Destructuring declaration	Unpack an object into variables through `componentN`	`val (id, name) = customer`
Destructuring in loops and maps	Works on `Pair`, map entries and data classes	`for ((city, orders) in byCity)`
`_` placeholder	Skip a component you do not need	`val (_, name) = customer`

Term	What it means	Example
`enum class`	Fixed set of constants, with properties and behaviour	`enum class Category(val tax: Int) { ELECTRONICS(18), BOOKS(0) }`
`valueOf` / `entries`	Look up by name (throws on unknown name) / all constants	`Category.valueOf("BOOKS")`, `Category.entries`
Immutability	Prefer `val` and read-only types; change state by creating a new object	`val order = Order(...)`
State transition	A function that returns a new object for the next state	`fun pay() = copy(status = PAID)`
Shallow copy trap	`copy()` shares the same inner list, so a mutable list can leak changes	Fix: store a read-only `List` and pass a new list
`sealed interface` / `sealed class`	Closed set of subtypes known at compile time; models states that differ in data	`sealed interface Shipment { object Packed; data class Shipped(val awb: String) }`
Exhaustive `when`	On a sealed type `when` needs no `else`; a missing branch is a compile error	`when (s) { is Packed -> ...; is Shipped -> ... }`
Enum vs sealed	Enum: same shape for every constant. Sealed: each state can carry its own data, impossible states cannot be built	`Shipped` has an `awb`, `Packed` does not
Primitive obsession	Using `Long` for every id so mix-ups compile	`addToCart(productId, orderId)` silently wrong
Value class `init`	Validate on creation	`init { require(value > 0) }`

5. Extensions, functions and lambdas

Term	What it means	Example
Extension function	Add a function to an existing type without editing it; replaces utility classes	`fun String.isCoupon() = startsWith("WELCOME")`
Receiver	The type before the dot in an extension, available as `this`	`fun Money.times(n: Int) = Money(amount * n, currency)`
Static dispatch	The declared type, not the runtime type, picks the extension	Extension on `Payment` is used even if the object is a `Card`
Member wins	If the class has a member with the same signature, the member is called	An extension named `toString()` is never used
Extension limits	Cannot read `private` members and cannot add state	No backing field in an extension
Extension property	Property with a getter only, no field	`val Order.isEmpty get() = items.isEmpty()`
Nullable receiver	Extension on `T?` is callable on a null value	`fun String?.orEmptyLabel() = this ?: "-"`
Calling extensions from Java	They compile to static methods taking the receiver first	`StringExtKt.isCoupon(s)`
`it`	Implicit name of a single lambda parameter	`list.map { it * 2 }`
Trailing lambda	A last lambda argument goes outside the parentheses	`items.filter { it.qty > 0 }`
Function reference	Pass an existing function with `::`	`names.map(String::uppercase)`
Generic function	Type parameter after `fun`	`fun <T> firstOr(list: List<T>, d: T): T`

6. Collections

Creating and changing

Term	What it means	Example
`listOf` / `setOf` / `mapOf`	Read-only collections	`val m = mapOf("A" to 1, "B" to 2)`
`mutableListOf` / `mutableSetOf` / `mutableMapOf`	Collections you can change	`val l = mutableListOf(1, 2); l.add(3)`
`buildList` / `buildMap`	Build with a block, get a read-only result	`buildList { add(1); addAll(other) }`
`to`	Makes a `Pair`, used for map entries	`"Laptop" to 65000`
`plus` / `minus` / `+=`	New collection with or without elements; `+=` on a `var` read-only list reassigns, on a `val` it does not compile	`var l = listOf(1); l += 2`
`emptyList()`	Empty read-only list	`val none = emptyList<String>()`

Transforming

Term	What it means	Example
`map`	Transform each element	`products.map { it.name }`
`filter` / `filterNot`	Keep or drop by a condition	`orders.filter { it.total > 1000 }`
`flatMap` / `flatten`	Map to lists and merge them into one	`orders.flatMap { it.items }`
`associateBy`	Map keyed by a property, value is the element	`products.associateBy { it.id }`
`associate` / `associateWith`	Build a map from pairs / use the element as key	`names.associateWith { it.length }`
`groupBy`	`Map<Key, List<Element>>`	`orders.groupBy { it.region }`
`mapValues`	Transform the values of a map	`grouped.mapValues { it.value.size }`
`eachCount`	Count per group (after `groupingBy`)	`words.groupingBy { it }.eachCount()`
`partition`	Split into two lists: matching and not matching	`val (cheap, dear) = items.partition { it.price < 100 }`
`sorted` / `sortedBy` / `sortedByDescending`	Return a new sorted list	`orders.sortedByDescending { it.total }`
`take` / `drop` / `first` / `last`	Pick parts of a list	`top.take(3)`
`distinct`	Remove duplicates	`cities.distinct()`

Aggregating

Term	What it means	Example
`sum` / `sumOf`	Total of numbers / total of a selector	`items.sumOf { it.qty * it.price }`
`count`	Number of elements, optionally matching	`orders.count { it.coupon != null }`
`average`	Mean of numbers	`totals.average()`
`fold`	Reduce with an initial value, safe on empty lists	`xs.fold(0) { acc, x -> acc + x }`
`reduce`	Reduce without an initial value, throws on empty	`xs.reduce { a, b -> a + b }`
`maxByOrNull` / `minByOrNull`	Element with the biggest or smallest selector, `null` if empty	`orders.maxByOrNull { it.total }`
`any` / `all` / `none`	Test the whole collection	`items.any { it.qty == 0 }`
`firstOrNull` / `find`	First match or `null`	`products.find { it.id == 3 }`

Pairs, sequences and views

Term	What it means	Example
`Pair` / `Triple`	Two or three values without a class	`val (name, price) = "Pen" to 20`
Destructuring in a lambda	Unpack a pair or map entry in the parameter list	`map.forEach { (city, total) -> println("$city $total") }`
Eager vs lazy	Collections run each step over all elements; sequences go element by element	`list.map{}.filter{}` vs `list.asSequence().map{}.filter{}`
`Sequence`	Lazy pipeline, good for big data or early exit	`generateSequence(1) { it * 2 }.take(5).toList()`
Intermediate vs terminal	Intermediate (`map`, `filter`) build the pipeline, a terminal (`toList`, `first`, `sum`) runs it	Nothing runs until the terminal call
`asSequence()` / `toList()`	Move between collection and sequence	`list.asSequence()...toList()`
Read-only is a view	`List` is not `ImmutableList`; the same list can change through another `MutableList` reference	Guard with `toList()` or private mutable plus public read-only
Defensive copy	Copy before exposing	`val items: List<Item> get() = _items.toList()`
One runtime type	`List` and `MutableList` are both `java.util.List` at runtime	Java can mutate a Kotlin read-only list
Streams vs Kotlin	Kotlin removes `.stream()` and `.collect(...)`; a stream is single use, a Kotlin list is reusable	`orders.map { it.total }` instead of `orders.stream().map(...).toList()`

7. Interop with Java and Spring

Build and framework plugins

Term	What it means	Example
`kotlin("jvm")` Gradle plugin	Adds Kotlin compilation to a Java Gradle build	`plugins { kotlin("jvm") version "..." }`
`src/main/kotlin`	Kotlin source folder; Kotlin compiles first and sees Java, Java sees the Kotlin output	One jar holds both languages
`kotlin-spring` (`plugin.spring`)	Opens classes with Spring annotations so proxies work (Kotlin classes are final)	`kotlin("plugin.spring")`
`allOpen`	Opens classes marked with chosen annotations	`allOpen { annotation("jakarta.persistence.Entity") }`
`kotlin-jpa` (`plugin.jpa`)	Generates the no-arg constructor JPA needs	`kotlin("plugin.jpa")`
Spring bean in Kotlin	Constructor injection through the primary constructor	`@RestController class C(private val service: CourseService)`

Kotlin calling Java

Term	What it means	Example
Platform type	Java type without nullability info; you choose `T` or `T?`	`val c: Course? = service.find(1)`
Nullable annotation understood	`@Nullable` Java returns become `T?`	`@Nullable Course find()`
SAM conversion	A lambda stands in for a Java single-method interface	`Collections.sort(list) { a, b -> a.compareTo(b) }`
Nullable request parameter	`String?` makes a Spring request parameter optional	`fun list(@RequestParam q: String?)`

JPA and configuration

Term	What it means	Example
Kotlin JPA entity	`@Entity` class with `var` fields, an `id` default and the JPA plugins	`@Entity class Course(var title: String, @Id @GeneratedValue val id: Long? = null)`
Entity `equals` / `hashCode`	Do not use `data class` for entities; use id-based `equals` and a constant `hashCode`	`override fun hashCode() = javaClass.hashCode()`
Spring Data repository	Derived queries in Kotlin	`fun findByTitleContaining(t: String): List<Course>`
`@ConfigurationProperties` data class	Typed settings with relaxed binding from `application.properties`	`@ConfigurationProperties("learnly") data class LearnlyProps(val title: String)`
`@ConfigurationPropertiesScan`	Registers properties classes on the application class	Without it the properties bean is missing

Java calling Kotlin

Term	What it means	Example
Generated getters and setters	Kotlin properties show up as accessors in Java; `archived` gives `getArchived()`	`p.getName()`, `p.setArchived(true)`
`INSTANCE`	Java reaches an `object` or companion through it	`CourseIds.INSTANCE.next()`
`@JvmStatic`	Generates a real static method	`@JvmStatic fun next()`
`@JvmOverloads`	Generates Java overloads for default arguments	`@JvmOverloads fun badge(t: String, s: String = "OPEN")`
`@JvmName` / `@file:JvmName`	Chooses the Java method name or the file's class name	`@file:JvmName("Formats")`
`@Throws`	Declares a checked exception so Java can catch it	`@Throws(IOException::class) fun load()`
`@JvmInline`	Marks a value class	`@JvmInline value class OrderId(val value: Long)`

8. Quick reference

Kotlin against Java

Task	Java	Kotlin
Constant	`final int x = 1;`	`val x = 1`
Variable	`int x = 1;`	`var x = 1`
Null-safe call	`a != null ? a.b() : null`	`a?.b()`
Default value	`a != null ? a : "n/a"`	`a ?: "n/a"`
Class with getters, equals, toString	POJO or `record`	`data class`
Static members	`static`	`companion object` / top-level / `@JvmStatic`
Singleton	private constructor and holder	`object`
Switch	`switch` / pattern `switch`	`when`
Utility class	`ShopUtils.format(x)`	`x.format()` (extension)
String building	`String.format(...)` / `+`	`"Hello $name"`
Filter and map	`list.stream().filter(...).map(...).toList()`	`list.filter { }.map { }`
Sealed hierarchy	`sealed interface`	`sealed interface` plus exhaustive `when`
Checked exceptions	`throws`	none required; `@Throws` for Java callers

Which scope function?

I want to...	Use
Run code only if not null	`x?.let { }`
Configure a new object and return it	`apply`
Log or side-effect inside a chain	`also`
Compute a result from members of an object	`run` / `with`

Common compile errors and what they mean

Message or symptom	Cause	Fix
`Val cannot be reassigned`	Assigned a `val` twice	Use `var`, or create a new value
`Null can not be a value of a non-null type String`	Put `null` in a non-null type	Make it `String?`
`Only safe (?.) or non-null asserted (!!.) calls are allowed`	Used a nullable value directly	Use `?.`, `?:` or a null check
`Smart cast to 'X' is impossible`	The value is a mutable property	Copy to a local `val`
`This type is final, so it cannot be inherited from`	Parent class not `open`	Add `open`
`'when' expression must be exhaustive`	A sealed or enum branch is missing	Add the branch (avoid `else` on sealed types)
`Type mismatch: Required OrderId, found Long`	Value class protects the id	Wrap the value: `OrderId(7)`
`Unresolved reference` on `+=` for a `val` list	`val` plus read-only `List` cannot be reassigned	Use `var`, or a `MutableList`
Spring: "class is final and cannot be proxied"	`kotlin-spring` plugin missing	Add `kotlin("plugin.spring")`
JPA: "No default constructor for entity"	`kotlin-jpa` plugin missing	Add `kotlin("plugin.jpa")`
Java: "non-static method cannot be referenced from a static context"	Calling an `object` member like a static method	Use `INSTANCE` or `@JvmStatic`

Gotchas to remember

- `val` is a read-only reference, not a deep freeze: a `val` pointing to a `MutableList` is still changeable.
- `List` is read-only, not immutable: it is a view and can change through another reference.
- `!!` is a crash waiting to happen; prefer `?.`, `?:` or `requireNotNull`.
- `data class` equality uses only the primary constructor properties; never use it for JPA entities.
- `copy()` is shallow: nested mutable objects are shared.
- Extensions are resolved statically and members always win.
- Sequences are lazy: nothing runs until a terminal operation.
- Java types are platform types: declare `T?` yourself when `null` is possible.
- Kotlin classes are final: Spring and JPA need `kotlin-spring`, `kotlin-jpa` or `allOpen`.